

Types of Databases

Database বিভিন্নভাবে classify করা যায়। বহুল ব্যবহৃত কয়েকটি Database type হলো:

1. Relational Database
2. NoSQL Database
3. Columnar Database
4. Graph Database
5. Key-Value Database

Relational Database

Relational Database হলো এমন একটি Database যেখানে Data-কে Table-এর মাধ্যমে organize করা হয়।

প্রতিটি Table-এ Data থাকে:

- **Rows**
- **Columns**

এবং বিভিন্ন Table-এর মধ্যে relationship তৈরি করা যায়।

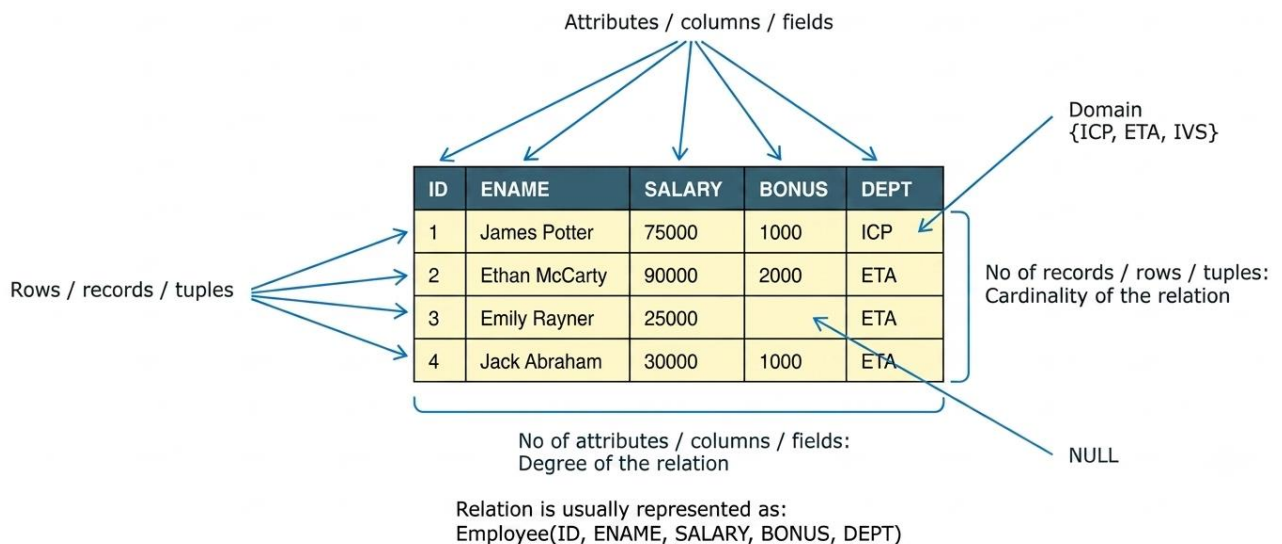
Relational Database-কে সাধারণভাবে **SQL Database**-ও বলা হয়।

Examples

- PostgreSQL
- MySQL
- Oracle
- Microsoft SQL Server
- SQLite

একটি Relational Table Employee Table:

	ID	ENAME	SALARY	BONUS	DEPT
1	James	Potter	75000	1000	ICP
2	Ethan	McCarty	90000	2000	ETA
3	Emily	Rayner	25000	NULL	ETA
4	Jack	Abraham	30000	1000	ETA



1. Row / Record / Tuple

Table-এর প্রতিটি **horizontal line** বা প্রতিটি individual record-কে বলা হয়:

- Row

- Record
- Tuple

উদাহরণ:

1 | James Potter | 75000 | 1000 | ICP

এটি একটি **Row/Record/Tuple**।

অর্থাৎ একজন Employee-এর সম্পূর্ণ information হলো একটি record।

সহজভাবে:

Row = একটি সম্পূর্ণ Record

2. Column / Attribute / Field

Table-এর **vertical অংশগুলো** হলো Column।

উদাহরণ:

ID
ENAME
SALARY
BONUS
DEPT

এগুলো প্রত্যেকটি একটি করে Column।

একই ধরনের information একটি Column-এ থাকে।

যেমন:

SALARY
75000
90000
25000
30000

সবগুলো Employee-এর Salary।

Column-এর আরও কিছু নাম:

- Attribute
- Field
- Column

এই তিনটি term অনেক ক্ষেত্রে একই অর্থে ব্যবহার করা হয়।

সহজভাবে:

Column = কোনো একটি নির্দিষ্ট ধরনের তথ্য/Attribute

3. Relation

Relational Database-এর একটি Table-কে mathematical/database terminology-তে **Relation** বলা হয়।

যেমন:

Employee
একটি Relation হতে পারে।
এবং Relation-এর মধ্যে থাকে:
Rows + Columns
অর্থাৎ:

Relation = Table

Database শেখার সময় "Relation" শব্দটি দেখলে প্রথমে **Table** হিসেবে বুঝতে পারো।

4. Degree of Relation

একটি Relation/Table-এ মোট কতগুলো **Column/Attribute** আছে, সেটাকে বলা হয়:

Degree of Relation

উপরের Employee Table-এ:

ID
ENAME
SALARY
BONUS
DEPT

মোট 5টি columns।

তাই:

Degree = 5

Formula:

Degree = Number of Columns / Attributes

5. Cardinality of Relation

একটি Table-এ মোট কতগুলো Rows/Records আছে, সেটাকে বলা হয়:

Cardinality of Relation

উপরের Table-এ:

- 1 → James Potter
- 2 → Ethan McCarty
- 3 → Emily Rayner
- 4 → Jack Abraham

মোট 4টি rows।

তাই:

Cardinality = 4

Formula:

Cardinality = Number of Rows / Records

Degree vs Cardinality

এটা ভালোভাবে বুঝে রাখবে।

Concept	কী count করে?	Example
Degree	Columns	5
Cardinality	Rows	4

Degree → Columns

Cardinality → Rows

6. Domain

Domain হলো কোনো Column-এ কী ধরনের/কোন কোন valid value থাকতে পারে তার নির্ধারিত set বা range।

উদাহরণ:

DEPT

Column-এর valid values হতে পারে:

{ICP, ETA, IVS}

অর্থাৎ Department column-এ এই নির্দিষ্ট ধরনের value গ্রহণ করা হবে।

আরেকটি সহজ example:

ধরো:

Gender

এর domain হতে পারে:

{Male, Female, Other}

অথবা:

Age

এর domain হতে পারে নির্দিষ্ট integer range।

সহজভাবে:

Domain = একটি Column-এ কী ধরনের valid value থাকতে পারে তার নির্ধারিত set/range।

7. NULL

Slide-এ BONUS column-এর একটি value দেখো:

Emily Rayner | 25000 | NULL | ETA

এখানে BONUS-এর value হলো:

NULL

NULL মানে 0 নয়।

এটা খুব গুরুত্বপূর্ণ।

NULL বলতে বোঝাতে পারে:

- Value জানা নেই
- Value দেওয়া হয়নি
- Value প্রযোজ্য নয়

Example:

BONUS = 0

মানে Employee-এর bonus শূন্য।

কিন্তু:

BONUS = NULL

মানে Bonus-এর information **missing/unknown/not available** হতে পারে।

SQL-এ NULL handling খুব গুরুত্বপূর্ণ হবে।

যেমন:

```
SELECT *
FROM employee
WHERE bonus IS NULL;
```

এখানে NULL খুঁজতে = NULL ব্যবহার করা হয় না।

2. NoSQL Database

NoSQL = Not Only SQL

NoSQL Database মূলত এমন Data handle করার জন্য ব্যবহৃত হয় যেখানে traditional relational table structure সবসময় সবচেয়ে উপযুক্ত নয়।

এগুলো বিশেষ করে:

- Semi-structured data
- Flexible data
- Large-scale applications
- Rapidly changing data

এর জন্য ব্যবহার করা হয়।

Example: MongoDB

ধরো একটি Student-এর information:

```
{
  "name": "Rahim",
  "age": 22,
  "skills": ["Python", "SQL", "ML"],
  "address": {
    "city": "Dhaka"
  }
}
```

এখানে প্রতিটি Student-এর জন্য একই ধরনের field থাকা বাধ্যতামূলক নয়।

একজনের skills থাকতে পারে, অন্যজনের নাও থাকতে পারে।

Relational Database-এ যেখানে predefined schema বেশি গুরুত্বপূর্ণ, NoSQL-এ অনেক ক্ষেত্রে **flexible schema** থাকে।

জনপ্রিয় NoSQL Database:

- MongoDB
- Redis
- Cassandra
- DynamoDB
-

3. Columnar Database

এখানে Data traditional row-wise না রেখে **column-oriented** ভাবে store করা হয়।

ধরো:

```
id    → 1, 2, 3, 4, 5
name  → A, B, C, D, E
salary → 20k, 30k, 40k, 50k, 60k
```

Traditional relational database সাধারণত row-oriented storage ব্যবহার করতে পারে।

Columnar Database-এর বড় সুবিধা হলো **analytical queries**-এর ক্ষেত্রে নির্দিষ্ট কিছু column খুব দ্রুত পড়তে পারে।

Example

ধরো তোমার কাছে ১০ কোটি employee records আছে। তুমি শুধু জানতে চাও:

Average salary কত?

তখন পুরো row পড়ার প্রয়োজন নেই। শুধু salary column-এর Data নিয়ে কাজ করলেই হবে।

এই কারণে Columnar Database/warehouse analytical workload-এর জন্য খুব useful।

Examples:

- Google BigQuery
- Amazon Redshift
- Snowflake
- ClickHouse

কোথায় ব্যবহার হয়?

মূলত:

- Data Warehouse
- Business Intelligence
- Analytics
- Large-scale Data Analysis

4. Graph Database

Graph Database এমন Data-এর জন্য useful যেখানে **relationships/connections** খুব গুরুত্বপূর্ণ।

এখানে মূল ধারণা:

Node + Relationship + Node

Example: Social Network

ধরো:

```
Rahim —FRIEND—> Karim
      |
      |—FOLLOWS—> Sumaiya
```

এখানে:

- Rahim → Node
- Karim → Node
- Sumaiya → Node
- FRIEND / FOLLOWS → Relationship

Facebook-এর মতো Social Network-এ: কে কার Friend? কে কাকে Follow করে? কোন মানুষের সঙ্গে কার connection

বেশি? এ ধরনের relationship-heavy problem-এ Graph Database খুব useful।

Examples:

- Neo4j
- Amazon Neptune

Recommendation System-এও ব্যবহার হতে পারে।

যেমন:

User → watches → Movie

User → likes → Movie

Movie → belongs to → Genre

এখান থেকে recommendation তৈরি করা যায়।

5. Key-Value Database

এখানে Data মূলত:

Key → Value

এই structure-এ রাখা হয়।

Example:

"user_101" → "Rahim"

"user_102" → "Karim"

"user_103" → "Sumaiya"

আরেকটি example:

"session_123" → "logged_in"

Key দিয়ে খুব দ্রুত Value retrieve করা যায়।

জনপ্রিয় Examples:

- Redis
- Amazon DynamoDB

কোথায় ব্যবহার হয়?

বিশেষ করে:

- Caching
- Session management
- Real-time applications
- Fast lookup

সবচেয়ে গুরুত্বপূর্ণ Comparison

Type	Data Structure	প্রধান ব্যবহার	Example
Relational	Tables	General business/data analysis	PostgreSQL
NoSQL	Documents/various	Flexible/semi-structured data	MongoDB
Columnar	Columns	Analytics/Data Warehouse	BigQuery
Graph	Nodes + Relationships	Connected data	Neo4j
Key-Value	Key → Value	Fast lookup/cache	Redis

ডেটাবেসের প্রকারভেদ

